

Chapter 1: Introduction to Formal Languages and Automata Theory

◆ 4 Marks Questions (Any 5)

Q1. Define Formal Language and Automata Theory.

Answer:

Formal Language and Automata Theory is the mathematical study of abstract machines (automata) and the problems they can solve. It focuses on the structure of languages and the computational models used to recognize and generate them.

Formal language refers to a set of strings formed from an alphabet according to specific rules. Automata theory helps us design models like Finite Automata, Pushdown Automata, and Turing Machines to represent and process these languages.

It forms the theoretical foundation of computer science, particularly for compilers, programming languages, and artificial intelligence.

Q2. What is an Alphabet? Give examples.

Answer:

An alphabet is a **non-empty finite set of symbols** used to form strings and languages. It is usually denoted by the Greek letter Σ (**Sigma**).

Each symbol in the alphabet is called a **character**.

Example:

- $\Sigma = \{a, b, c\} \rightarrow$ alphabet of letters
- $\Sigma = \{0, 1\} \rightarrow$ binary alphabet
- $\Sigma = \{x, y, z\} \rightarrow$ symbolic alphabet

Alphabets are the building blocks of formal languages, as all strings and expressions are made using these symbols.

Q3. Define a String and its Length with an example.

Answer:

A **string** is a finite sequence of symbols taken from an alphabet Σ . The number of symbols in a string is called its **length**, denoted by $|w|$.

Example:

If $\Sigma = \{a, b\}$, possible strings are: a, ab, bba, etc.

- For $w = "ab"$, $|w| = 2$

- For $w = \text{"abba"}$, $|w| = 4$

The **empty string**, represented by ϵ , has a length of zero ($|\epsilon| = 0$).

Strings are used to form languages, and operations like concatenation, reversal, and substring apply to them.

Q4. Define Prefix, Suffix, and Substring with examples.

Answer:

- **Prefix:** A prefix of a string is any leading part of the string.
Example: For $w = \text{"TOC"}$, prefixes are ϵ , T, TO, TOC.
 - **Suffix:** A suffix of a string is any trailing part of it.
Example: For $w = \text{"TOC"}$, suffixes are ϵ , C, OC, TOC.
 - **Substring:** Any consecutive sequence of symbols from the string.
Example: For $w = \text{"TOC"}$, substrings are T, O, C, TO, OC, TOC.
Prefixes and suffixes are always substrings, but not all substrings are prefixes or suffixes.
-

Q5. What is an Empty Language and Non-Empty Language? Give examples.

Answer:

- **Empty Language (Φ):** A language that does not contain any string, even the empty string, is called an empty language.
Example: $L = \{\}$ or $L = \Phi$.
 - **Non-Empty Language:** A language that contains at least one string is called non-empty.
Example: $L = \{a, ab, bba\}$.
Empty and non-empty languages are the simplest forms of formal languages used to build larger and more complex language structures.
-
-

◆ 5 Marks Questions (Any 5)

Q1. Explain the Chomsky Classification of Formal Languages.

Answer:

According to Noam Chomsky, formal languages are classified into **four types** based on their generative power:

1. **Type 0 – Recursively Enumerable Languages:**
Generated by unrestricted grammar; recognized by **Turing Machine**.
 2. **Type 1 – Context-Sensitive Languages:**
Generated by context-sensitive grammar; recognized by **Linear Bounded Automata (LBA)**.
 3. **Type 2 – Context-Free Languages:**
Generated by context-free grammar; recognized by **Pushdown Automata (PDA)**.
 4. **Type 3 – Regular Languages:**
Generated by regular grammar; recognized by **Finite Automata (FA)**.
This classification helps understand which computational model can recognize a particular type of language.
-

Q2. Explain the different types of Automata.**Answer:**

Automata are mathematical models used to describe the behavior of machines that recognize formal languages. The main types are:

1. **Finite Automata (FA):** Accepts regular languages; simplest model.
 2. **Pushdown Automata (PDA):** Uses a stack memory; accepts context-free languages.
 3. **Linear Bounded Automata (LBA):** A Turing machine with limited tape; accepts context-sensitive languages.
 4. **Turing Machine (TM):** Most powerful automata model; can simulate any computation.
- Each automaton increases in computational power from FA to TM.
-

Q3. What is a Grammar? Explain its components with example.**Answer:**

Grammar is a set of rules used to generate strings in a language.

It is denoted as $G = (V, T, P, S)$, where:

- **V:** Set of variables or non-terminal symbols.

- **T:** Set of terminal symbols.
- **P:** Set of production rules.
- **S:** Start symbol.

Example:

$G = (\{A, X, Y, Z\}, \{a, b, c\}, P, A)$

$P = \{A \rightarrow XYZ, X \rightarrow a, Y \rightarrow b, Z \rightarrow c\}$

The string generated is “abc”. Grammar defines how valid strings of a language can be derived.

Q4. Explain the concept of Formal Language with examples.

Answer:

A **formal language** is a collection of strings formed using an alphabet and governed by specific syntactic rules.

It may restrict which strings are allowed.

Example:

If $\Sigma = \{0, 1\}$, possible languages are:

- $L1 = \{00, 11, 01, 10\}$
- $L2 = \{(01)^n \mid n \geq 1\}$
- $L3 = \{0^n 1^m \mid n, m \geq 1\}$

Formal languages are the foundation of programming and machine languages.

Q5. Write short notes on Recursive and Non-Recursive Grammar.

Answer:

- **Recursive Grammar:**

A grammar is recursive if a variable appears on both sides of the production rule.

Example: $S \rightarrow aSb \mid \epsilon$

Here, S appears on both sides, hence recursive.

- **Non-Recursive Grammar:**

No variable appears on both sides of a production rule.

Example: $S \rightarrow ab \mid \epsilon$

Recursive grammars can generate infinite strings, while non-recursive grammars generate finite sets.

◆ 6 Marks Questions (Any 5)

Q1. Explain the concept of Language and its types with examples.

Answer:

A **language** is a collection of strings formed from an alphabet (Σ). It defines the valid combinations of symbols accepted in a system.

Types of Languages:

1. **Empty Language:** Contains no strings. Example: $L = \Phi$.
2. **Non-Empty Language:** Contains one or more strings. Example: $L = \{a, ab\}$.
3. **Finite Language:** Contains a finite number of strings. Example: $L = \{0, 01, 011\}$.
4. **Infinite Language:** Contains infinitely many strings. Example: $L = \{0^n 1^n \mid n \geq 1\}$.

Languages are essential for defining computational problems and machine design.

Q2. (a) Define substring, prefix, and suffix.

(b) Differentiate between trivial and non-trivial substrings.

(c) Give formulas related to substrings.

Answer:

(a) Substring: Any consecutive part of a string. Example: $TOC \rightarrow TO, OC$.

Prefix: Starting sequence (ϵ, T, TO).

Suffix: Ending sequence (ϵ, C, OC).

(b) Trivial Substring: The string itself and ϵ .

Non-Trivial Substring: Any substring other than the string itself and ϵ .

(c) Formulas:

- No. of substrings = $n(n+1)/2$
 - No. of trivial substrings = 2
 - No. of non-trivial substrings = $n(n+1)/2 - 2$
- These properties help in string processing and pattern matching.
-

Q3. Explain Chomsky's classification of Grammars with examples.

Answer:

1. **Type 0 – Unrestricted Grammar:**

- Production: $\alpha \rightarrow \beta$, where α and β are any combination of symbols.
- Example: $S \rightarrow aSb \mid \epsilon$

2. Type 1 – Context-Sensitive Grammar:

- Rule: $\alpha A \beta \rightarrow \alpha \gamma \beta$ (context affects replacement).
- Example: $aAb \rightarrow abb$

3. Type 2 – Context-Free Grammar:

- Rule: $A \rightarrow \beta$ (single non-terminal on LHS).
- Example: $S \rightarrow aSb \mid \epsilon$

4. Type 3 – Regular Grammar:

- Rule: $A \rightarrow aB$ or $A \rightarrow a$
- Example: $S \rightarrow aA, A \rightarrow b$

Higher types generate all lower types. Each grammar type corresponds to a specific automaton.

Q4. (a) Define Automata and Expressive Power.

(b) Explain the expressive power of different automata.

Answer:

(a) **Automata:** A mathematical model that represents computation.

It defines how a machine transitions between states to process input strings.

(b) **Expressive Power:** The number or type of languages an automaton can recognize.

Automata	Model	Expressive Power
FA	Finite Automata	1 (Regular Languages)
PDA	Pushdown Automata	2 (Context-Free Languages)
LBA	Linear Bounded Automata	3 (Context-Sensitive Languages)
TM	Turing Machine	4 (Recursive Enumerable)

Thus, Turing Machine has the highest expressive power among all automata.

Q5. Explain Derivation and Parse Tree with example.

Answer:

Derivation:

It is the step-by-step process of generating a string using grammar rules.

Each step replaces a non-terminal with its production.

Example:

Grammar:

$A \rightarrow XYZ$

$X \rightarrow a, Y \rightarrow b, Z \rightarrow c$

Derivation:

$A \Rightarrow XYZ \Rightarrow aYZ \Rightarrow abZ \Rightarrow abc$

Parse Tree:

A graphical representation showing how a string is derived.

A
/
|
\
X Y Z
a b c

Derivation trees help visualize how strings conform to grammar rules.

Chapter 2: Regular Languages and Automata Theory

◆ 4 Marks Questions (Any 5)

Q1. Define Regular Expression and give examples.

Answer:

A **Regular Expression (R.E.)** is a symbolic representation used to describe a set of strings (regular language) formed over an alphabet using specific operators such as **+**, **.**, and *****.

It provides a compact way to represent patterns and define regular languages.

Example:

- $r = (a + b)^*$ represents all strings of a's and b's, including ϵ .
- $r = ab(a + b)^*$ represents all strings starting with "ab".

Regular expressions are fundamental in defining search patterns, lexical analyzers, and finite automata.

Q2. What are Regular Operators? Mention their precedence.

Answer:

Regular Operators are special symbols used in regular expressions to combine alphabets and subexpressions.

There are **three main operators**:

1. **Union (+):** Represents logical OR operation.
Example: $(a + b) \rightarrow$ either 'a' or 'b'.
2. **Concatenation (.):** Joins two expressions.
Example: $ab \rightarrow$ 'a' followed by 'b'.
3. **Kleene Closure (*):** Represents repetition of zero or more occurrences.
Example: $a^* \rightarrow \epsilon, a, aa, aaa, \dots$

Operator Precedence:

Kleene Closure (*) $\rightarrow$ Concatenation (.) $\rightarrow$ Union (+)

Q3. Write any four identities of Regular Expressions.

Answer:

Some important identities used in simplifying regular expressions are:

1. $(r^*)^* = r^*$
2. $r^* \cdot r = r^+$
3. $r^* + r^+ = r^*$
4. $(r_1 + r_2)^* = (r_1^* + r_2^*)^* = (r_1^* \cdot r_2^*)^*$
5. If $r = \Phi$, then $r^* = \epsilon$ and $r^+ = \Phi$

These identities are helpful in minimizing and simplifying regular expressions for efficient computation.

Q4. Define DFA (Deterministic Finite Automata).

Answer:

A **Deterministic Finite Automaton (DFA)** is a mathematical model that accepts or rejects strings based on input symbols and state transitions.

A DFA is represented as a 5-tuple:

$(Q, \Sigma, \delta, q_0, F)$

Where,

- $Q \rightarrow$ finite set of states
- $\Sigma \rightarrow$ alphabet
- $\delta \rightarrow$ transition function ($Q \times \Sigma \rightarrow Q$)
- $q_0 \rightarrow$ initial state
- $F \rightarrow$ final/accepting state(s)

Each input leads to only one unique transition from a state, hence deterministic.

Q5. Define NFA and differentiate it from DFA.

Answer:

A **Non-Deterministic Finite Automata (NFA)** is a finite automaton where for a given state and input symbol, there may be **more than one possible transition**.

Formally, NFA is represented as:

$(Q, \Sigma, \delta, q_0, F)$ where $\delta: Q \times \Sigma \rightarrow 2^Q$

Difference between DFA and NFA:

Feature	DFA	NFA
Transition	Single next state	Multiple next states

Feature	DFA	NFA
ϵ -transitions	Not allowed	Allowed
Determinism	Deterministic	Non-deterministic
Complexity	Easy to simulate	Easy to design

◆ 5 Marks Questions (Any 5)

Q1. Explain the process of constructing a DFA with an example.

Answer:

Steps to construct a DFA:

1. **Recognize the language:** Understand the requirement (start, end, substring).
2. **Find minimum string:** Identify the smallest string that must be accepted.
3. **Create states:** Represent each input and transition step using circles.
4. **Satisfy all inputs:** Each state must handle all symbols from Σ .
5. **Mark final states:** Indicate acceptance with double circles.

Example: DFA accepting strings starting with “a” over $\Sigma = \{a, b\}$.

- $L = \{a, ab, abb, \dots\}$
 - Minimum string = “a”.
 - Construct transitions so that $q_0 \rightarrow q_1$ on ‘a’, and all other strings starting with b go to a trap state.
-

Q2. Explain the equivalence between Regular Expressions and Finite Automata.

Answer:

Regular Expressions (R.E.) and Finite Automata (FA) are equivalent in computational power.

This means that:

1. For every regular expression, there exists a finite automaton that accepts the same language.

2. For every finite automaton, there exists a regular expression that defines the same language.

Hence,

$$L(R) = L(F)$$

Where,

- $L(R)$ = language generated by R.E.
- $L(F)$ = language accepted by Finite Automata.

This equivalence is used in compiler design, lexical analysis, and pattern recognition.

Q3. What are the types of Regular Expressions? Explain.

Answer:

Regular Expressions are classified into three types:

1. **Restricted (Standard) R.E.:** Uses only +, ., and * operators. Example: $(a + b)^*$
2. **Semi-Restricted R.E.:** Includes intersection ($\cap$). Example: $(a + b)^* \cap (a + c)^*$
3. **Unrestricted R.E.:** Includes complement ($\sim$) and intersection ($\cap$). Example: $\sim(a + b)^*$

These variations help define more complex language properties in formal systems.

Q4. Write the steps to simplify a Regular Expression with an example.

Answer:

Simplification of a regular expression involves applying identities to minimize it.

Steps:

1. Apply idempotent law: $(r + r = r)$.
2. Use absorption law: $(r + r.s = r)$.
3. Apply closure properties: $(r^*)^* = r^*$.
4. Combine similar terms.

Example: Simplify $(0 + \epsilon)^*$.

$$\rightarrow (0^* + \epsilon)^*$$

$$\rightarrow (0^*)^*$$

$$\rightarrow 0^*$$

Hence, simplified expression = 0^* .

Q5. Explain Regular Grammar and its relation to Finite Automata.

Answer:

A **Regular Grammar** is a set of production rules that generate regular languages.

It is represented as **$G = (V, T, S, P)$** .

- V: Non-terminal symbols
- T: Terminal symbols
- S: Start symbol
- P: Production rules of the form $A \rightarrow aB$ or $A \rightarrow a$

Relation with FA:

- Every Regular Grammar can be converted to a Finite Automata.
 - Every FA can be represented by an equivalent Regular Grammar.
- Thus, Regular Grammar $\Leftrightarrow$ Finite Automata $\Leftrightarrow$ Regular Expression.
-
-

◆ **6 Marks Questions (Any 5)**

Q1. (a) Define DFA formally.

(b) Explain its components with an example.

(c) What are the acceptance conditions?

Answer:

(a) DFA = $(Q, \Sigma, \delta, q_0, F)$

(b)

- $Q \rightarrow \{q_0, q_1, q_2, \dots\}$ finite set of states
- $\Sigma \rightarrow \{\text{input symbols}\}$
- $\delta \rightarrow$ Transition function
- $q_0 \rightarrow$ Start state
- $F \rightarrow$ Final states

Example: DFA accepting strings ending with 'b'.

- $\Sigma = \{a, b\}$, $Q = \{q_0, q_1\}$, $q_0 \rightarrow q_1$ on 'b'.

- $F = \{q_1\}$.
- (c) **Acceptance:**
 A string is accepted if $\delta^*(q_0, w) \in F$.
 Otherwise, it is rejected.

Q2. Explain the difference between DFA and NFA with a neat table.

Answer:

Feature	DFA	NFA
Transition Function	Single next state ($\delta: Q \times \Sigma \rightarrow Q$)	Multiple next states ($\delta: Q \times \Sigma \rightarrow 2^Q$)
ϵ -Transitions	Not allowed	Allowed
Determinism	Deterministic	Non-Deterministic
Ease of Design	Harder	Easier
Simulation	Easy	Needs conversion to DFA
Acceptance	Single path to final state	At least one path to final state

Though DFA and NFA differ in structure, both accept **the same set of regular languages**.

Q3. Explain Closure Properties of Regular Languages.

Answer:

Regular Languages are closed under several operations, meaning performing these operations results in another regular language.

Closure Operations:

1. **Union** ($L_1 \cup L_2$)
2. **Intersection** ($L_1 \cap L_2$)
3. **Complement** ($\sim L$)
4. **Difference** ($L_1 - L_2$)

5. **Concatenation ($L_1.L_2$)**

6. **Reversal (L^R)**

7. **Kleene Closure (L^*)**

8. **Positive Closure (L^+)**

Example:

If $L_1 = \{a, b\}$ and $L_2 = \{b\}$, then $L_1 \cup L_2 = \{a, b\}$.

These properties prove the mathematical robustness of regular languages.

Q4. (a) What is Pumping Lemma?

(b) State its conditions.

(c) Use it to show that $L = \{0^n 1^n \mid n \geq 1\}$ is not regular.

Answer:

(a) **Pumping Lemma:** Used to prove that a language is **not regular**.

(b) **Conditions:**

If A is a regular language, then there exists a number p (pumping length) such that for any string s in A with $|s| \geq p$,

$s = xyz$ satisfies:

1. $xy^i z \in A$ for all $i \geq 0$

2. $|y| > 0$

3. $|xy| \leq p$

(c) **Example:**

Assume $L = \{0^n 1^n \mid n \geq 1\}$ is regular.

Take $s = 0^p 1^p$ and divide as $x = 0^k$, $y = 0^m$, $z = \text{remaining}$.

Pumping y will make more 0's than 1's, violating the pattern $0^n 1^n$.

Hence, L is **not regular**.

Q5. Explain the process of Minimization of Finite Automata.

Answer:

DFA minimization reduces the number of states without changing the language it accepts.

Steps:

1. **Remove unreachable states.**

2. **Create a transition table** for all states.

3. **Divide states into final and non-final sets (P_0).**

4. **Refine partitions** by checking distinguishability of states ($P_1, P_2 \dots$).
5. **Stop** when no further partitioning is possible.
6. **Merge equivalent states** to form the minimized DFA.
The resulting automaton is the simplest possible DFA that accepts the same language.

.

Chapter 2: Regular Languages and Automata Theory

◆ 4 Marks Questions (Any 5)

Q1. Define Regular Expression and give examples.

Answer:

A **Regular Expression (R.E.)** is a symbolic representation used to describe a set of strings (regular language) formed over an alphabet using specific operators such as **+**, **.**, and *****.

It provides a compact way to represent patterns and define regular languages.

Example:

- $r = (a + b)^*$ represents all strings of a's and b's, including ϵ .
- $r = ab(a + b)^*$ represents all strings starting with "ab".

Regular expressions are fundamental in defining search patterns, lexical analyzers, and finite automata.

Q2. What are Regular Operators? Mention their precedence.

Answer:

Regular Operators are special symbols used in regular expressions to combine alphabets and subexpressions.

There are **three main operators**:

1. **Union (+):** Represents logical OR operation.
Example: $(a + b) \rightarrow$ either 'a' or 'b'.
2. **Concatenation (.):** Joins two expressions.
Example: $ab \rightarrow$ 'a' followed by 'b'.
3. **Kleene Closure (*):** Represents repetition of zero or more occurrences.
Example: $a^* \rightarrow \epsilon, a, aa, aaa, \dots$

Operator Precedence:

Kleene Closure (*) $\rightarrow$ Concatenation (.) $\rightarrow$ Union (+)

Q3. Write any four identities of Regular Expressions.

Answer:

Some important identities used in simplifying regular expressions are:

1. $(r^*)^* = r^*$
2. $r^* \cdot r = r^+$
3. $r^* + r^+ = r^*$
4. $(r_1 + r_2)^* = (r_1^* + r_2^*)^* = (r_1^* \cdot r_2^*)^*$
5. If $r = \Phi$, then $r^* = \epsilon$ and $r^+ = \Phi$

These identities are helpful in minimizing and simplifying regular expressions for efficient computation.

Q4. Define DFA (Deterministic Finite Automata).

Answer:

A **Deterministic Finite Automaton (DFA)** is a mathematical model that accepts or rejects strings based on input symbols and state transitions.

A DFA is represented as a 5-tuple:

$(Q, \Sigma, \delta, q_0, F)$

Where,

- $Q \rightarrow$ finite set of states
- $\Sigma \rightarrow$ alphabet
- $\delta \rightarrow$ transition function ($Q \times \Sigma \rightarrow Q$)
- $q_0 \rightarrow$ initial state
- $F \rightarrow$ final/accepting state(s)

Each input leads to only one unique transition from a state, hence deterministic.

Q5. Define NFA and differentiate it from DFA.

Answer:

A **Non-Deterministic Finite Automata (NFA)** is a finite automaton where for a given state and input symbol, there may be **more than one possible transition**.

Formally, NFA is represented as:

$(Q, \Sigma, \delta, q_0, F)$ where $\delta: Q \times \Sigma \rightarrow 2^Q$

Difference between DFA and NFA:

Feature	DFA	NFA
Transition	Single next state	Multiple next states

Feature	DFA	NFA
ϵ -transitions	Not allowed	Allowed
Determinism	Deterministic	Non-deterministic
Complexity	Easy to simulate	Easy to design

◆ 5 Marks Questions (Any 5)

Q1. Explain the process of constructing a DFA with an example.

Answer:

Steps to construct a DFA:

1. **Recognize the language:** Understand the requirement (start, end, substring).
2. **Find minimum string:** Identify the smallest string that must be accepted.
3. **Create states:** Represent each input and transition step using circles.
4. **Satisfy all inputs:** Each state must handle all symbols from Σ .
5. **Mark final states:** Indicate acceptance with double circles.

Example: DFA accepting strings starting with “a” over $\Sigma = \{a, b\}$.

- $L = \{a, ab, abb, \dots\}$
 - Minimum string = “a”.
 - Construct transitions so that $q_0 \rightarrow q_1$ on ‘a’, and all other strings starting with b go to a trap state.
-

Q2. Explain the equivalence between Regular Expressions and Finite Automata.

Answer:

Regular Expressions (R.E.) and Finite Automata (FA) are equivalent in computational power.

This means that:

1. For every regular expression, there exists a finite automaton that accepts the same language.

2. For every finite automaton, there exists a regular expression that defines the same language.

Hence,

$$L(R) = L(F)$$

Where,

- $L(R)$ = language generated by R.E.
- $L(F)$ = language accepted by Finite Automata.

This equivalence is used in compiler design, lexical analysis, and pattern recognition.

Q3. What are the types of Regular Expressions? Explain.

Answer:

Regular Expressions are classified into three types:

1. **Restricted (Standard) R.E.:** Uses only +, ., and * operators. Example: $(a + b)^*$
2. **Semi-Restricted R.E.:** Includes intersection ($\cap$). Example: $(a + b)^* \cap (a + c)^*$
3. **Unrestricted R.E.:** Includes complement ($\sim$) and intersection ($\cap$). Example: $\sim(a + b)^*$

These variations help define more complex language properties in formal systems.

Q4. Write the steps to simplify a Regular Expression with an example.

Answer:

Simplification of a regular expression involves applying identities to minimize it.

Steps:

1. Apply idempotent law: $(r + r = r)$.
2. Use absorption law: $(r + r.s = r)$.
3. Apply closure properties: $(r^*)^* = r^*$.
4. Combine similar terms.

Example: Simplify $(0 + \epsilon)^*$.

$$\rightarrow (0^* + \epsilon)^*$$

$$\rightarrow (0^*)^*$$

$$\rightarrow 0^*$$

Hence, simplified expression = 0^* .

Q5. Explain Regular Grammar and its relation to Finite Automata.

Answer:

A **Regular Grammar** is a set of production rules that generate regular languages.

It is represented as **$G = (V, T, S, P)$** .

- V: Non-terminal symbols
- T: Terminal symbols
- S: Start symbol
- P: Production rules of the form $A \rightarrow aB$ or $A \rightarrow a$

Relation with FA:

- Every Regular Grammar can be converted to a Finite Automata.
 - Every FA can be represented by an equivalent Regular Grammar.
- Thus, Regular Grammar $\Leftrightarrow$ Finite Automata $\Leftrightarrow$ Regular Expression.
-
-

◆ **6 Marks Questions (Any 5)**

Q1. (a) Define DFA formally.

(b) Explain its components with an example.

(c) What are the acceptance conditions?

Answer:

(a) DFA = $(Q, \Sigma, \delta, q_0, F)$

(b)

- $Q \rightarrow \{q_0, q_1, q_2, \dots\}$ finite set of states
- $\Sigma \rightarrow \{\text{input symbols}\}$
- $\delta \rightarrow$ Transition function
- $q_0 \rightarrow$ Start state
- $F \rightarrow$ Final states

Example: DFA accepting strings ending with 'b'.

- $\Sigma = \{a, b\}$, $Q = \{q_0, q_1\}$, $q_0 \rightarrow q_1$ on 'b'.

- $F = \{q_1\}$.
- (c) **Acceptance:**
 A string is accepted if $\delta^*(q_0, w) \in F$.
 Otherwise, it is rejected.

Q2. Explain the difference between DFA and NFA with a neat table.

Answer:

Feature	DFA	NFA
Transition Function	Single next state ($\delta: Q \times \Sigma \rightarrow Q$)	Multiple next states ($\delta: Q \times \Sigma \rightarrow 2^Q$)
ϵ -Transitions	Not allowed	Allowed
Determinism	Deterministic	Non-Deterministic
Ease of Design	Harder	Easier
Simulation	Easy	Needs conversion to DFA
Acceptance	Single path to final state	At least one path to final state

Though DFA and NFA differ in structure, both accept **the same set of regular languages**.

Q3. Explain Closure Properties of Regular Languages.

Answer:

Regular Languages are closed under several operations, meaning performing these operations results in another regular language.

Closure Operations:

1. **Union** ($L_1 \cup L_2$)
2. **Intersection** ($L_1 \cap L_2$)
3. **Complement** ($\sim L$)
4. **Difference** ($L_1 - L_2$)

5. **Concatenation ($L_1.L_2$)**

6. **Reversal (L^R)**

7. **Kleene Closure (L^*)**

8. **Positive Closure (L^+)**

Example:

If $L_1 = \{a, b\}$ and $L_2 = \{b\}$, then $L_1 \cup L_2 = \{a, b\}$.

These properties prove the mathematical robustness of regular languages.

Q4. (a) What is Pumping Lemma?

(b) State its conditions.

(c) Use it to show that $L = \{0^n 1^n \mid n \geq 1\}$ is not regular.

Answer:

(a) **Pumping Lemma:** Used to prove that a language is **not regular**.

(b) **Conditions:**

If A is a regular language, then there exists a number p (pumping length) such that for any string s in A with $|s| \geq p$,

$s = xyz$ satisfies:

1. $xy^i z \in A$ for all $i \geq 0$

2. $|y| > 0$

3. $|xy| \leq p$

(c) **Example:**

Assume $L = \{0^n 1^n \mid n \geq 1\}$ is regular.

Take $s = 0^p 1^p$ and divide as $x = 0^k$, $y = 0^m$, $z = \text{remaining}$.

Pumping y will make more 0's than 1's, violating the pattern $0^n 1^n$.

Hence, L is **not regular**.

Q5. Explain the process of Minimization of Finite Automata.

Answer:

DFA minimization reduces the number of states without changing the language it accepts.

Steps:

1. **Remove unreachable states.**

2. **Create a transition table** for all states.

3. **Divide states into final and non-final sets (P_0).**

4. **Refine partitions** by checking distinguishability of states ($P_1, P_2 \dots$).
5. **Stop** when no further partitioning is possible.
6. **Merge equivalent states** to form the minimized DFA.
The resulting automaton is the simplest possible DFA that accepts the same language.

Chapter 4 – Pushdown Automata (PDA) and Context-Free Languages (CFLs)

◆ 4 Marks Questions (Any 5)

Q1. Define Pushdown Automata (PDA).

Answer:

A **Pushdown Automata (PDA)** is an automaton that uses a **stack memory** to recognize **Context-Free Languages (CFLs)**.

It is represented as a 7-tuple:

$(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ where:

- $Q \rightarrow$ Set of states
- $\Sigma \rightarrow$ Input alphabet
- $\Gamma \rightarrow$ Stack alphabet
- $\delta \rightarrow$ Transition function ($Q \times \Sigma \times \Gamma \rightarrow P(Q \times \Gamma^*)$)
- $q_0 \rightarrow$ Start state
- $Z_0 \rightarrow$ Initial stack symbol
- $F \rightarrow$ Final states

PDAs are more powerful than finite automata since they can handle nested structures like parentheses and recursion.

Q2. What is a Context-Free Grammar (CFG)?

Answer:

A **Context-Free Grammar (CFG)** generates **context-free languages (CFLs)**, which are accepted by a PDA.

Formally, a CFG is represented as $G = (V, T, S, P)$ where:

- $V \rightarrow$ Non-terminal symbols
- $T \rightarrow$ Terminal symbols
- $S \rightarrow$ Start symbol
- $P \rightarrow$ Productions of the form $A \rightarrow \alpha$, where $A \in V$ and $\alpha \in (V \cup T)^*$.

Example:

$S \rightarrow aSb \mid \epsilon$ generates strings like $ab, aabb, aaabbb$, etc.

CFLs are used in compilers, programming languages, and syntax analysis.

Q3. Define Deterministic and Non-Deterministic PDA.

Answer:

- **Deterministic PDA (DPDA):**

For each combination of state, input, and stack symbol, there is **at most one transition**.

DPDA accepts a subset of CFLs.

- **Non-Deterministic PDA (NPDA):**

Allows **multiple transitions** for the same input and stack symbol.

NPDA can recognize all context-free languages.

Thus, **$DPDA \subset NPDA$** .

Q4. Define Instantaneous Description (ID) of PDA.

Answer:

An **Instantaneous Description (ID)** represents the current configuration of a PDA.

It is written as (q, w, α) where:

- $q \rightarrow$ current state
- $w \rightarrow$ remaining input
- $\alpha \rightarrow$ stack contents (top at left).

Example: (q_0, ab, Z_0) means PDA is in state q_0 , has input “ab” left, and stack top is Z_0 .

The ID changes with each transition according to δ .

Q5. Define acceptance by final state and empty stack in PDA.

Answer:

A PDA can accept input in two ways:

1. **Acceptance by Final State:**

If the PDA reaches a final state after reading the input.

2. **Acceptance by Empty Stack:**

If the stack becomes empty after processing the input.

Both methods are equivalent, meaning if a PDA accepts by one method, another PDA can be constructed to accept by the other.

◆ 5 Marks Questions (Any 5)

Q1. Explain the 7-tuple definition of PDA with example.

Answer:

A PDA is represented by $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.

Example: PDA for $L = \{a^n b^n \mid n \geq 1\}$.

- $Q = \{q_0, q_1, q_2\}, \Sigma = \{a, b\}, \Gamma = \{Z_0, A\}$
 - $q_0 = \text{start}, F = \{q_2\}$
 - δ transitions:
 - $\delta(q_0, a, Z_0) = (q_0, AZ_0)$
 - $\delta(q_0, a, A) = (q_0, AA)$
 - $\delta(q_0, b, A) = (q_1, \epsilon)$
 - $\delta(q_1, b, A) = (q_1, \epsilon)$
 - $\delta(q_1, \epsilon, Z_0) = (q_2, Z_0)$

This PDA pushes A for each 'a' and pops for each 'b', ensuring equal count.
-

Q2. Explain the relationship between CFG and PDA.

Answer:

CFG and PDA are **equivalent** in recognizing context-free languages.

- Every CFG can be converted into an equivalent PDA.
- Every PDA has an equivalent CFG that generates the same language.

Conversion Idea:

For each production $A \rightarrow \alpha$ in CFG, the PDA pushes α onto the stack when A is on top.

Hence, $L(G) = L(\text{PDA})$.

This relationship is used in parser construction for compilers.

Q3. Differentiate between Regular and Context-Free Languages.

Answer:

Aspect	Regular Languages	Context-Free Languages
Recognized by	Finite Automata	Pushdown Automata
Grammar Type	Regular Grammar (Type 3)	Context-Free Grammar (Type 2)
Memory	No stack	Stack used
Example	$(a + b)^*ab$	$a^n b^n$
Applications	Pattern matching, lexing	Syntax analysis, parsing

Q4. What are the applications of Context-Free Grammars?

Answer:

CFGs are widely used in computer science for:

1. **Syntax Analysis:** Parsing programming languages.
2. **Compiler Design:** Used by syntax analyzers to check structure.
3. **Expression Evaluation:** For mathematical and logical expressions.
4. **Natural Language Processing:** Modeling sentence structure.
5. **XML and HTML Parsing:** To validate tag structures.

Hence, CFGs are essential in both artificial and natural language processing.

Q5. Explain acceptance by empty stack with example.

Answer:

A PDA accepts input if the stack becomes empty after reading the input.

Example: $L = \{a^n b^n \mid n \geq 1\}$

- Initially, stack has Z_0 .
- Push 'A' for each 'a'.
- Pop 'A' for each 'b'.

If input ends and stack is empty, PDA accepts.

This ensures equal number of a's and b's.

◆ 6 Marks Questions (Any 5)

Q1. (a) Define PDA formally.

(b) Construct a PDA for $L = \{a^n b^n \mid n \geq 1\}$.

Answer:

(a) PDA = $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

(b) Steps:

1. Push A for each 'a'.
2. Pop A for each 'b'.
3. Accept if stack empty.

Transitions:

- $\delta(q_0, a, Z_0) = (q_0, AZ_0)$
- $\delta(q_0, a, A) = (q_0, AA)$
- $\delta(q_0, b, A) = (q_1, \epsilon)$
- $\delta(q_1, b, A) = (q_1, \epsilon)$
- $\delta(q_1, \epsilon, Z_0) = (q_f, Z_0)$

This PDA accepts equal number of a's and b's.

Q2. Explain CFG to PDA conversion with steps.

Answer:

Steps:

1. Start with CFG $G = (V, T, S, P)$.
2. Construct PDA $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$.
3. Push S (start symbol) to stack.
4. For each production $A \rightarrow \alpha$, replace A (top of stack) with α .
5. When a terminal matches input, pop it.
6. Accept when input exhausted and stack empty.

This process proves equivalence between CFG and PDA.

Q3. Explain Deterministic and Non-Deterministic PDA with examples.

Answer:

DPDA: Has one possible move for each state-input-stack combination.

Example: $L = \{a^n b^n\}$.

NPDA: May have multiple moves; handles ambiguous grammars.

Example: $L = \{a^n b^m c^n \mid n, m \geq 1\}$.

$DPDA \subset NPDA$ since non-determinism allows broader language recognition.

Q4. Explain Instantaneous Description (ID) and moves in PDA.

Answer:

An **Instantaneous Description** (q, w, α) represents:

- $q \rightarrow$ Current state
- $w \rightarrow$ Unread input
- $\alpha \rightarrow$ Stack contents

Moves:

$(q, aw, X\beta) \vdash (p, w, \gamma\beta)$ if $\delta(q, a, X) = (p, \gamma)$.

Example:

$(q_0, ab, Z_0) \vdash (q_0, b, AZ_0) \vdash (q_1, \varepsilon, \varepsilon)$

Describes step-by-step PDA computation.

Q5. Write the differences between DFA and PDA.

Answer:

Aspect	DFA	PDA
Memory	No memory	Uses stack memory
Language Type	Regular Language	Context-Free Language
Transitions	$\delta: Q \times \Sigma \rightarrow Q$	$\delta: Q \times \Sigma \times \Gamma \rightarrow P(Q \times \Gamma^*)$
Power	Less powerful	More powerful
Example	$(a + b)^*ab$	$a^n b^n$

Thus, PDA extends DFA's capabilities by adding memory via stack.

Chapter 5 – Turing Machine (TM) and Linear Bounded Automata (LBA)

◆ 4 Marks Questions (Any 5)

Q1. Define a Turing Machine.

Answer:

A **Turing Machine (TM)** is a mathematical model of computation capable of simulating any algorithm.

It consists of:

1. **Input tape:** Infinite memory divided into cells.
2. **Tape head:** Reads/writes symbols and moves left or right.
3. **State register:** Holds the current state.

TM is represented as $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$.

It recognizes **Recursively Enumerable Languages (Type-0)**.

Q2. What is the difference between FA, PDA, and TM?

Answer:

Feature	FA	PDA	TM
Memory	None	Stack	Infinite tape
Language	Regular	Context-Free	Recursive Enumerable
Power	Lowest	Moderate	Highest
Example	$(a + b)^*$	$a^n b^n$	Palindromes

Thus, TM is the most powerful computational model.

Q3. Define Transition Function of Turing Machine.

Answer:

The **transition function δ** defines how a TM moves from one configuration to another.

$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$

Where,

- $Q \rightarrow$ Set of states
- $\Gamma \rightarrow$ Tape symbols
- $L, R \rightarrow$ Head movement (Left or Right)

Example:

$\delta(q_0, 1) = (q_1, X, R) \rightarrow$ Replace 1 with X and move right.

Q4. Define Linear Bounded Automata (LBA).

Answer:

A **Linear Bounded Automaton (LBA)** is a restricted Turing Machine whose tape is limited to the length of the input.

It recognizes **Context-Sensitive Languages (CSLs)**.

Formally, $LBA = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$.

The input acts as both input and memory, so it cannot expand indefinitely.

Q5. What are the features of a Turing Machine?

Answer:

1. **Single infinite tape** divided into cells.
 2. **Tape head** for reading and writing symbols.
 3. **Finite set of states.**
 4. **Transition function** governs movements.
 5. **Can perform calculations and simulate algorithms.**
It is the most powerful automaton model in the Chomsky hierarchy.
-
-

◆ 5 Marks Questions (Any 5)

Q1. Explain components of a Turing Machine.

Answer:

TM has three main components:

1. **Tape:** Infinite sequence of cells used for input and storage.
2. **Tape Head:** Reads and writes symbols, moves left/right.
3. **Finite Control:** Decides next state using δ .

Example:

$$\delta(q_0, 1) = (q_1, X, R).$$

Thus, TM reads input, modifies it, and decides acceptance.

Q2. Explain acceptance and rejection in Turing Machine.

Answer:

- **Acceptance:** If TM enters the accepting state (q_{accept}), the input is accepted.
- **Rejection:** If TM enters the rejecting state (q_{reject}), input is rejected.

If TM never halts, input is undefined.

Hence,

$$L(M) = \{w \mid M \text{ accepts } w\}$$

This halting behavior defines computability.

Q3. Explain the working of Turing Machine with example.

Answer:

Example: TM to recognize $L = \{0^n 1^n \mid n \geq 1\}$.

1. Replace 0 with X $\rightarrow$ move right.
2. Replace 1 with Y $\rightarrow$ move left.
3. Repeat until all symbols replaced.
4. Accept if all replaced, reject otherwise.

This TM simulates checking equal numbers of 0s and 1s.

Q4. Explain the role of Linear Bounded Automata in language recognition.

Answer:

LBA recognizes **Context-Sensitive Languages (CSLs)** where production rules are of the form $\alpha A \beta \rightarrow \alpha \gamma \beta$.

It has limited memory, i.e., the tape cannot exceed input length.

Example:

Language $L = \{a^n b^n c^n \mid n \geq 1\}$ can be recognized by LBA but not PDA.

Thus, LBA bridges the gap between PDA and TM in computational power.

Q5. Explain the relationship between LBA and Turing Machine.

Answer:

LBA is a **restricted form of TM**:

Aspect	LBA	TM
Tape Size	Limited to input	Infinite
Language Type	Context-Sensitive	Recursive Enumerable
Power	Less	More
Example	$a^n b^n c^n$	Palindromes

Thus, TM can simulate LBA, but not vice versa.

◆ **6 Marks Questions (Any 5)**

Q1. (a) Define Turing Machine.

(b) Design a TM for palindrome checking.

Answer:

(a) $TM = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$.

(b) Steps:

1. Replace first and last characters with X and check equality.
2. Move head inward each time.
3. If mismatch $\rightarrow$ reject; if all matched $\rightarrow$ accept.

TM thus identifies palindromic strings like “abba”, “aba”.

Q2. Compare FA, PDA, LBA, and TM.

Answer:

Model	Memory	Language	Example
FA	None	Regular	$(a + b)^*ab$
PDA	Stack	Context-Free	$a^n b^n$
LBA	Limited Tape	Context-Sensitive	$a^n b^n c^n$
TM	Infinite Tape	Recursive Enumerable	Palindromes

This hierarchy shows the increasing power of computation.

Q3. Explain types of Turing Machines.

Answer:

1. **Deterministic TM:** One transition per state-symbol pair.
 2. **Non-Deterministic TM:** Multiple transitions allowed.
 3. **Multi-Tape TM:** Uses several tapes for parallel operations.
 4. **Universal TM:** Can simulate any other TM.
 5. **Halting TM:** Always stops after finite steps.
- Each variation extends TM's capability and efficiency.

Q4. Explain Chomsky hierarchy in terms of automata.

Answer:

Type	Language	Grammar	Automata
Type 0	Recursive Enumerable	Unrestricted	Turing Machine
Type 1	Context-Sensitive	Context-Sensitive	LBA
Type 2	Context-Free	CFG	PDA

Type	Language	Grammar	Automata
Type 3	Regular	Regular Grammar	FA

This hierarchy shows how each automaton corresponds to a grammar type.

Q5. Explain Halting Problem in Turing Machine.

Answer:

The **Halting Problem** states that it is **impossible** to determine whether a Turing Machine will halt for a given input.

Alan Turing proved that no algorithm can solve this universally.

It's a key concept in computability theory and defines limits of machine intelligence.

Chapter 6 – Decidability and Undecidability

◆ 4 Marks Questions (Any 5)

Q1. Define Decidable Language.

Answer:

A **Decidable Language** is a language for which there exists a **Turing Machine (TM)** that halts and gives a **yes/no** answer for every input string.

If the string belongs to the language, the TM accepts; otherwise, it rejects.

In other words, the TM always halts on all inputs.

Example:

$L = \{w \mid w \text{ contains equal number of 0s and 1s}\}$ is decidable since a TM can count and halt.

Q2. Define Undecidable Problem.

Answer:

An **Undecidable Problem** is a decision problem that **no Turing Machine can solve** for all possible inputs.

The TM either loops infinitely or fails to give a correct answer for some cases.

Example: The **Halting Problem** is undecidable — no algorithm can determine whether a TM halts for every input.

Q3. What is a Decision Problem?

Answer:

A **Decision Problem** is a question with a **yes/no answer**, defined for all possible inputs.

If a TM always halts and gives an answer, the problem is **decidable**; otherwise, it's **undecidable**.

Example:

“Does string w belong to language L ?”

If TM halts and answers $\rightarrow$ decidable.

If TM runs forever $\rightarrow$ undecidable.

Q4. Define Recursive and Recursively Enumerable Languages.

Answer:

- **Recursive Language (Decidable):** TM halts on every input (accept/reject).
- **Recursively Enumerable Language (RE):** TM halts **only on accepted strings**, may loop for others.

Relation:

Every Recursive language is RE, but not every RE language is Recursive.

Recursive $\subset$ Recursively Enumerable

Q5. What is the Halting Problem?

Answer:

The **Halting Problem** asks: “Will a Turing Machine M halt on input w?”

Alan Turing proved that no general algorithm can decide this.

Hence, it is an **undecidable problem**.

It demonstrates the limits of computation — some problems cannot be solved by any algorithm.

◆ 5 Marks Questions (Any 5)

Q1. Explain Decidable and Undecidable Languages with examples.

Answer:

- **Decidable Language:** There exists a TM that halts for all inputs.
Example: $L = \{0^n 1^n \mid n \geq 0\}$.
- **Undecidable Language:** No TM can halt and give correct result for all inputs.
Example: The Halting Problem.
Decidability defines whether a problem can be solved algorithmically; undecidability shows limitations of TMs.

Q2. Explain Recursive and Recursively Enumerable Languages with examples.

Answer:

- **Recursive (Decidable):** TM halts on all inputs.
Example: $L = \{w \mid w \text{ has even number of 0s}\}$.
- **Recursively Enumerable (Semi-Decidable):** TM halts only on accepted strings; loops on others.

Example: $L = \{\langle M, w \rangle \mid M \text{ halts on } w\}$.

Thus, Recursive $\subset$ RE.

Recursive = Decidable; RE = Semi-decidable.

Q3. What is the Post Correspondence Problem (PCP)?

Answer:

PCP is an **undecidable problem** where we are given two lists of strings:

$A = \{a_1, a_2, \dots, a_n\}$, $B = \{b_1, b_2, \dots, b_n\}$.

The problem asks whether there exists a sequence $i_1, i_2, \dots, i_k$ such that

$a_{i_1}a_{i_2}\dots a_{i_k} = b_{i_1}b_{i_2}\dots b_{i_k}$.

It was proved by **Emil Post** to be undecidable.

PCP helps demonstrate the boundaries of algorithmic solvability.

Q4. Explain the difference between Semi-Decidable and Decidable Problems.

Answer:

Feature	Decidable	Semi-Decidable
TM Halting	Always halts	Halts only on accept
Output	Yes/No	May loop on reject
Example	$L = \{0^n1^n\}$	Halting Problem

Hence, decidable problems have total algorithms; semi-decidable problems have partial algorithms.

Q5. Explain closure properties of Recursive and RE languages.

Answer:

Recursive and RE languages are closed under certain operations:

Operation	Recursive RE	
Union	Closed	Closed

Operation	Recursive RE	
Intersection	Closed	Closed
Complement	Closed	Not closed
Concatenation	Closed	Closed
Kleene Star	Closed	Closed

This shows recursive languages have stronger closure properties than RE languages.

◆ 6 Marks Questions (Any 5)

Q1. Explain the concept of Decidability in detail with examples.

Answer:

A problem is **decidable** if there exists a TM that halts for all inputs and gives a definite yes/no answer.

Examples:

1. Testing if a DFA accepts an empty language.
 2. Checking if two DFAs accept the same language.
- These problems are decidable because algorithms can simulate FA transitions and halt with results.
- Thus, decidability ensures algorithmic solvability.
-

Q2. Explain the Halting Problem and prove it is undecidable.

Answer:

Statement: No TM can determine whether another TM halts for a given input.

Proof (by contradiction):

Assume TM H decides halting problem.

$H(\langle M, w \rangle)$:

- Accept if M halts on w

- Reject if M doesn't halt

Now construct TM D that runs H on $\langle M, M \rangle$ and reverses result.

If H exists, D behaves paradoxically $\rightarrow$ contradiction.

Hence, **no such H exists** $\rightarrow$ halting problem is undecidable.

Q3. Explain the Post Correspondence Problem (PCP) with example.

Answer:

Given two lists A and B:

$A = \{1, 10, 011\}$, $B = \{101, 0, 11\}$.

Find sequence making strings equal.

E.g., $i_1 = 2$, $i_2 = 3$ gives:

A: $10 + 011 = 10011$

B: $0 + 11 = 011 \rightarrow$ not equal

No sequence works $\rightarrow$ unsolvable.

Hence, PCP is **undecidable**.

PCP shows there exist problems no TM can solve.

Q4. Discuss closure properties of Recursive and RE languages.

Answer:

Recursive Languages (R):

Closed under Union, Intersection, Complement, Concatenation, and Kleene Star.

Recursively Enumerable (RE):

Closed under Union, Intersection, Concatenation, and Kleene Star but **not under Complement**.

This difference arises because non-halting TMs break complement closure.

Hence, $R \subset RE$, and closure under complement is the main distinction.

Q5. Explain the relationship between Decidability and Computability.

Answer:

Decidability refers to whether a problem can be solved algorithmically.

Computability studies whether a function can be calculated by a TM.

- **Decidable Problems:** Computable functions that halt.

- **Undecidable Problems:** Non-computable functions.

Example: Addition $\rightarrow$ computable; Halting problem $\rightarrow$ non-computable.

Hence, computability is the foundation, and decidability is its practical subset.

Chapter 7 – Applications of Automata Theory and Computability

◆ 4 Marks Questions (Any 5)

Q1. What are the real-world applications of Automata Theory?

Answer:

Automata theory is used in:

1. **Compiler Design:** Tokenization, syntax parsing.
 2. **Text Searching:** Used in regex engines.
 3. **Networking:** Protocol verification.
 4. **Artificial Intelligence:** State machines in robotics.
 5. **Software Verification:** Model checking.
It provides mathematical models for real computation and logic.
-

Q2. How is Automata used in Compiler Design?

Answer:

Automata theory forms the foundation for:

- **Lexical Analysis:** Finite Automata for token recognition.
 - **Syntax Analysis:** Pushdown Automata for parsing.
 - **Code Generation:** Based on grammar and language rules.
Hence, compilers convert source code to machine code using automata concepts.
-

Q3. What is the role of Regular Expressions in programming languages?

Answer:

Regular Expressions (RE) describe patterns of strings.

They are used in:

- **Search & Replace operations** (e.g., grep).
- **Input validation** (emails, phone numbers).

- **Token generation** in compilers.

Thus, RE connects theory (Regular Languages) with real-world computing tasks.

Q4. Explain the use of Pushdown Automata in parsing.

Answer:

PDAs are used to check **syntax structure** in programming.

They handle nested constructs like parentheses, loops, and recursion.

Example: Grammar of “if-else” or “while” statements.

The stack in PDA tracks nested levels of statements, making it ideal for parsing.

Q5. What is the importance of the Chomsky Hierarchy in computer science?

Answer:

Chomsky Hierarchy classifies languages and automata:

- Type 3 → Regular (FA)
- Type 2 → Context-Free (PDA)
- Type 1 → Context-Sensitive (LBA)
- Type 0 → Recursively Enumerable (TM)

This hierarchy shows computational power progression and is crucial in language design and compiler construction.

◆ 5 Marks Questions (Any 5)

Q1. Explain applications of Finite Automata.

Answer:

1. **Lexical Analysis:** DFA used in tokenization.
2. **Text Processing:** Used in regex matchers.
3. **Control Systems:** State transitions.
4. **Protocol Analysis:** Network communication validation.
5. **Speech Recognition:** Word pattern matching.

Finite automata provide efficient, deterministic solutions in these areas.

Q2. Explain the role of Context-Free Grammar in programming languages.

Answer:

CFGs describe the **syntax** of programming languages.

Used in:

- **Parsing expressions** (e.g., arithmetic, conditional).
- **Syntax checking.**
- **Compiler front-end.**

Example:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T \times F \mid F$$
$$F \rightarrow \text{id} \mid (E)$$

CFG ensures hierarchical and nested language structure representation.

Q3. Explain the use of Turing Machines in modern computation.

Answer:

Turing Machines model general-purpose computers.

Applications:

1. **Algorithm Design:** Abstract model for logic.
 2. **Artificial Intelligence:** Learning algorithms.
 3. **Theoretical Computer Science:** Computability analysis.
 4. **Complexity Theory:** Defining P, NP classes.
TMs define limits of what computers can and cannot compute.
-

Q4. Explain Automata's role in Artificial Intelligence.

Answer:

Automata provide a **state-based model** for decision-making systems.

Applications:

- **Game design:** Finite state machines for NPC behavior.
- **Chatbots:** Pattern-based response control.
- **Robotics:** Motion planning.
Automata simplify logical flow representation in AI systems.

Q5. Explain how Automata Theory helps in software verification.

Answer:

Automata-based **model checking** verifies that software meets its specifications.

Steps:

1. Convert software model to automaton.
2. Define required properties (safety, liveness).
3. Use algorithms to check property satisfaction.
If property fails, counterexample provided.
This ensures **reliable and error-free** software.

◆ **6 Marks Questions (Any 5)**

Q1. (a) Explain major applications of Automata Theory.

(b) Describe one in detail.

Answer:

(a) Applications:

1. Compiler Design
2. Text Search
3. AI Systems
4. Software Verification
5. Network Protocol Design

(b) Compiler Design:

Finite Automata recognize tokens, PDA parses syntax, and grammar ensures semantic correctness.

Automata thus power all compiler phases from source to executable.

Q2. Explain real-life uses of Regular Expressions and Finite Automata.

Answer:

Regular Expressions:

Used in text editors, search tools, and input validation.

Finite Automata:

Implements regex internally, providing deterministic state transitions for pattern detection.

Example: Regex “a*b” → DFA accepting “b”, “ab”, “aab”, etc.

Used in text filters and compiler lexers.

Q3. Explain the relationship between Automata and Compiler Design.

Answer:

Compiler Phase	Automata Model
Lexical Analysis	Finite Automata
Syntax Analysis	Pushdown Automata
Semantic Analysis	CFG + Stack Checking
Code Generation	Grammar Rules

Thus, automata form the core of compiler construction.

Q4. Explain the role of Automata Theory in Artificial Intelligence and Robotics.

Answer:

In AI and Robotics, automata model **state transitions** for autonomous decisions.

Examples:

- **FSMs in robots:** Decide motion based on environment.
 - **Pattern learning systems:** Use regular languages for recognition.
 - **Chatbots:** PDA-like models for grammar-based replies.
- Hence, automata support logical reasoning in machines.
-

Q5. Explain future trends and research applications of Automata Theory.

Answer:

1. **Quantum Automata:** Integration with quantum computing.
2. **DNA Computing:** Automata models for genetic pattern analysis.
3. **AI Algorithms:** State-machine-based intelligent control.

4. **Cybersecurity:** Automata for anomaly detection.

Automata continues to expand beyond theory into real-world intelligent systems.

Chapter 1 – Introduction to Formal Languages and Automata Theory

1. Write all possible strings of length ≤ 3 over the alphabet $\Sigma = \{a, b\}$.
 2. Find the number of substrings (trivial + non-trivial) of the string “TOC”.
 3. Construct a language L over $\Sigma = \{0, 1\}$ that contains all strings beginning and ending with 1.
 4. Determine whether the string “abba” belongs to $L = \{w \mid w \text{ has equal number of } a \text{ and } b\}$.
 5. Represent the relation between prefix, suffix, and substring of “COMPUTER”.
-

✿ Chapter 2 – Regular Languages and Finite Automata

1. Construct a DFA for $L = \{w \mid w \text{ starts with } a \text{ and ends with } b, \Sigma = \{a, b\}\}$.
 2. Convert the following NFA to equivalent DFA:
States = $\{q_0, q_1\}$, q_0 start, q_1 final; Transitions: $q_0 \xrightarrow{a} q_0$, $q_0 \xrightarrow{b} q_1$, $q_1 \xrightarrow{a} q_1$.
 3. Write a regular expression for all strings over $\{0, 1\}$ ending with 01.
 4. Construct a minimal DFA for $L = (0 + 1)^* 01$.
 5. Use the Pumping Lemma to show $L = \{0^n 1^n \mid n \geq 1\}$ is not regular.
-

✿ Chapter 3 – Grammar and Properties of Regular Languages

1. Construct a Right Linear Grammar for $L = (a + b)^* ab$.
 2. Convert the given FA to Regular Grammar with transitions: $q_0 \xrightarrow{a} q_1$, $q_1 \xrightarrow{b} q_2$, q_2 final.
 3. Find whether the language $L = \{a^m b^n \mid m < n\}$ is regular using Pumping Lemma.
 4. Write the derivation and parse tree for $S \rightarrow aSb \mid \varepsilon$ when the string is aabb.
 5. Determine if $L = (a + b)^* a (a + b)^*$ is closed under reversal.
-

✿ Chapter 4 – Pushdown Automata and Context-Free Languages

1. Construct a PDA for $L = \{a^n b^n \mid n \geq 1\}$.
2. Convert the following CFG to PDA: $S \rightarrow aSb \mid \varepsilon$.

3. Show by construction that $L = \{a^n b^m \mid n \neq m\}$ is not accepted by a Deterministic PDA.
 4. Design a CFG for all palindromes over $\Sigma = \{a, b\}$.
 5. Find the Instantaneous Description (ID) sequence for input “aabb” in a PDA for $L = \{a^n b^n \mid n \geq 1\}$.
-

Chapter 5 – Turing Machine and Linear Bounded Automata

1. Construct a Turing Machine to accept $L = \{0^n 1^n \mid n \geq 1\}$.
 2. Design a TM to decide whether an input string over $\{0, 1\}$ is a palindrome.
 3. Construct a TM that replaces all 0s with 1s on the tape.
 4. Give the transition table of a TM for $L = \{w \mid w \text{ contains at least one } 0\}$.
 5. Construct a Linear Bounded Automata for $L = \{a^n b^n c^n \mid n \geq 1\}$.
-

Chapter 6 – Decidability and Undecidability

1. Show that the Halting Problem for Turing Machines is undecidable.
 2. Construct an example of a language that is Recursively Enumerable but not Recursive.
 3. Determine whether the following problem is decidable: “Given two DFA M_1 and M_2 , does $L(M_1) = L(M_2)$?”
 4. Write an instance of Post Correspondence Problem (PCP) that has no solution.
 5. Verify the closure of Recursive Languages under complementation using examples.
-

Chapter 7 – Applications and Computational Concepts

1. Design a Finite Automaton for a traffic-light controller with states {Red, Green, Yellow}.
2. Construct a regular expression for validating email addresses (assume simplified pattern).
3. Model an automatic vending machine using Finite State Machine.

4. Write a Context-Free Grammar for simple arithmetic expressions ($E \rightarrow E + T \mid T$; $T \rightarrow T \times F \mid F$; $F \rightarrow \text{id} \mid (E)$).
5. Design a Finite Automata for binary strings divisible by 3.